Chapter 50: Memory Management

1. Overview

Angular applications are long-lived, stateful single-page applications. Unlike a traditional multi-page site where a full navigation wipes the JavaScript heap clean, an Angular SPA can stay open in a browser tab for hours or days — through dozens of route changes, hundreds of component creations and destructions, and thousands of async events. Every object an Angular component creates that outlives the component itself — an RxJS subscription, a native `addEventListener`, a `setInterval` timer, a reference held by a singleton service, a third-party widget instance — is a candidate for a **memory leak**.

A memory leak in a browser context is not "the garbage collector is broken." The V8/JS garbage collector (GC) is correct: it will never collect an object that is still *reachable* from a GC root (window, document, active closures, global variables, timers). Angular memory leaks are therefore almost always **reachability leaks** — something keeps a reference alive after the component/view that "owns" it has been logically destroyed. The DOM node is detached from `document`, but JavaScript still holds a pointer to it, so the GC cannot reclaim it. Over time, hundreds of detached component trees pile up, DOM node counts climb, subscription counts climb, and the tab eventually degrades (jank, slow change detection, OOM crash on mobile).

This chapter covers the concrete Angular-specific leak sources, the modern prevention APIs (`takeUntilDestroyed`, `DestroyRef`, `async` pipe), how to diagnose leaks with Chrome DevTools heap snapshots, and how to use `WeakMap` / `WeakRef` correctly when you genuinely need a component-keyed cache that must not itself become a leak source.

2. Core Concepts

2.1 Why Angular components leak: the reachability model

The JS engine only frees memory that is **unreachable** from the root set. Angular's `ComponentRef.destroy()` removes the component's view from the DOM and runs `ngOnDestroy`, but it does **not** magically null out every reference anyone else holds to that component instance, its injector, or objects it created. If any of the following still exist after destroy, the whole object graph reachable from them (which can include the entire component, its template, child components, and DOM subtree) stays in memory:

- An active `Subscription` to an `Observable` that a **service** (not the component) created and holds onto (e.g., inside `.subscribe()` called from the component, subscribing to a long-lived `Subject` in a singleton service).
- A DOM event listener attached via `element.addEventListener` or `Renderer2.listen` that was never removed.
- A `setInterval` / `setTimeout` handle whose callback closure captures `this` (the component).
- A reference stored in a shared/singleton service's array/map/Subject that was never removed (`this.registry.push(this)`).
- A third-party library instance (chart, map, rich-text editor, video player) that keeps internal references to DOM nodes/callbacks and has no explicit `destroy()` / `dispose()` call.

2.2 The classic leak sources catalogue

1. Unmanaged RxJS subscriptions

```
ngOnInit() {
  this.dataService.stream$.subscribe(v => this.value = v); // never unsubscribed
}
```

If `dataService` is a root-provided singleton and `stream$` is built from a `Subject` / `interval` / `fromEvent` that lives as long as the app, the subscription callback closure keeps `this` (the component) alive forever, even after route navigation destroys the component's view.

2. Native/Renderer2 event listeners without cleanup

```
ngOnInit() {
  window.addEventListener('resize', this.onResize); // leak: window never dies
}
// missing: ngOnDestroy() { window.removeEventListener('resize', this.onResize); }
```

`window` and `document` are permanent GC roots. Any listener registered on them is a permanent reference chain into your component unless explicitly removed.

3. Detached DOM nodes retained by closures

A node becomes "detached" when Angular removes it from the visible DOM tree (structural directive `*ngIf` toggling off, route change) but a JS variable/closure still references it — commonly via a manually cached `ElementRef`, a jQuery/D3 selection stored in a service, or an event handler closure that captured the element. DevTools heap snapshots flag these explicitly as "Detached HTMLDivElement" etc.

4. Third-party library handles

Charting libraries (Chart.js, ECharts), map libraries (Leaflet, Mapbox, Google Maps), CKEditor/TinyMCE, video players — many hold internal timers, resize observers, and DOM listeners. If the library exposes `.destroy()` / `.dispose()`, it must be called in `ngOnDestroy`. Some libraries have historically had incomplete cleanup APIs, requiring manual listener removal on top of calling `.destroy()`.

5. Uncanceled `setInterval` / `setTimeout`

```
ngOnInit() {
  this.timerId = setInterval(() => this.poll(), 1000);
}
ngOnDestroy() {
  clearInterval(this.timerId); // required — otherwise timer keeps firing and keeping `this`
alive
}
```

Timers are GC roots (managed by the browser's task queue). A live interval's callback closure keeps everything it references alive indefinitely, and it keeps executing against a destroyed component (wasted CPU, and if it touches `@ViewChild` refs, can throw or silently do nothing useful).

6. Services holding references to destroyed components

```
@Injectable({ providedIn: 'root' })
class RegistryService {
  activeWidgets: WidgetComponent[] = [];
  register(w: WidgetComponent) { this.activeWidgets.push(w); }
  // no unregister call anywhere
}
```

Because the service is root-scoped (lives for the app's lifetime), the array holds strong references forever unless the component explicitly removes itself in `ngOnDestroy`.

7. Closures over `this` in output/callback registrations passed to non-Angular APIs
Passing `this.handleClick.bind(this)` (a new function each time, cannot be removed with `removeEventListener` unless you keep the exact reference) or an inline arrow function to a native API is a subtle variant of #2 — you cannot even clean it up later because you never stored the reference to the exact function that was added.

2.3 Prevention patterns

`takeUntil` + a `Subject` (classic pre-Angular-16 pattern)

```
private destroy$ = new Subject<void>();
ngOnInit() {
  this.dataService.stream$.pipe(takeUntil(this.destroy$)).subscribe(...);
}
ngOnDestroy() {
  this.destroy$.next();
  this.destroy$.complete();
}
```

Works, but boilerplate-heavy and easy to forget on one of several subscriptions.

`takeUntilDestroyed()` (Angular 16+, RxJS interop package)
Automatically ties unsubscription to the component/directive's `DestroyRef`. No manual `Subject` bookkeeping. Must be called either in an injection context (constructor/field initializer) or given an explicit `DestroyRef`.

`DestroyRef.onDestroy()`
The generalized primitive underlying `takeUntilDestroyed`. Lets you register *any* cleanup callback (not just RxJS) against the component/directive/embedded-view destruction lifecycle — ideal for native listeners, third-party widget teardown, timers.

`async` pipe
Best default for template-bound observables: the pipe subscribes on `ngOnInit`/creation and calls `unsubscribe()` in its own `ngOnDestroy`, tied to the enclosing view's lifecycle. Removes the need for manual subscription management entirely for display-only streams.

Manual `unsubscribe()` in `ngOnDestroy`
Still valid for one-off subscriptions, especially in older codebases or where `DestroyRef` isn't in scope. Store `Subscription` objects, ideally combined via `Subscription.add()` or a `Subscription` array, and call `.unsubscribe()` once in `ngOnDestroy`.

2.4 Detecting leaks with heap snapshots

Chrome DevTools → Memory tab → "Heap snapshot":

1. Load the app, navigate to the suspect view, navigate away (destroy it), force GC (trash-can icon in Memory panel), take snapshot A.
2. Repeat the navigate-in/navigate-out cycle N times (e.g., 5–10), force GC again, take snapshot B.
3. Use the **Comparison** view between A and B — objects whose count grew proportionally to the number of repetitions (not just once) are leak candidates.
4. Filter the constructor list for your component class name or `Detached` (Chrome groups detached DOM trees under "Detached HTMLDivElement" etc. in newer versions, or you filter by typing the tag name and check "Distance"/retainers).
5. Click an instance → inspect **Retainers** panel (bottom) → walk up the reference chain until you hit a GC root (e.g., `Window`, a `Subject`'s `observers` array, a service's array field). That retainer chain is your leak's root cause.

A three-snapshot pass (idle → after N iterations → after N more iterations, with forced GC between each) is the standard technique to separate "still-settling" caches from genuine unbounded growth — real leaks show linear growth across all three snapshots.

2.5 WeakMap / WeakRef in Angular

- `WeakMap<ComponentRef, Metadata>` — useful for associating auxiliary bookkeeping data with a component instance from a service **without** preventing the component from being GC'd once all other strong references disappear. Keys must be objects (component instances, `ElementRef.nativeElement`, DOM nodes) — never primitives.
- `WeakRef` — lets you hold a reference to an object (e.g., a DOM node or component instance) that can still be collected; you must check `.deref()` for `undefined` before use. Useful for building your own caches, or observing an object's lifetime (e.g., debugging tools, custom pooling) without becoming the leak yourself.
- `FinalizationRegistry` — pairs with `WeakRef` to run a callback when an object is actually collected; useful for **detecting** leaks in dev tooling (log if a component's finalizer never fires within N seconds of navigation) — not a substitute for proper cleanup, and not guaranteed to run promptly/at all (spec allows non-deterministic timing), so never rely on it for correctness (e.g., don't release a resource only in a finalizer).

These are diagnostic/optimization tools, not a replacement for calling `unsubscribe()` / `removeEventListener` / `destroy()`. Using `WeakMap` to "fix" a leak that's actually caused by an active subscription or timer is a red herring — the timer/subscription closure is still a *strong* reference regardless of what data structure you used elsewhere.

3. Code Examples

3.1 `takeUntilDestroyed` usage

```
import { Component, inject } from '@angular/core';
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';
import { interval } from 'rxjs';
```

```
@Component({
  selector: 'app-live-ticker',
  template: `<span>{{ tick }}</span>`,
})
export class LiveTickerComponent {
  private priceService = inject(PriceService); // singleton, long-lived stream
  tick = 0;

  constructor() {
    // Must be called in an injection context (constructor / field initializer)
    // because it needs to grab the current DestroyRef implicitly.
    this.priceService.priceUpdates$
      .pipe(takeUntilDestroyed())
      .subscribe(price => (this.tick = price));
  }
}
```

Outside an injection context (e.g., inside `ngOnInit`, or in a plain class/service), pass the `DestroyRef` explicitly:

```
import { Component, DestroyRef, inject, OnInit } from '@angular/core';
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';

@Component({ selector: 'app-report', template: `` })
export class ReportComponent implements OnInit {
  private destroyRef = inject(DestroyRef); // captured while injection context is valid
  private reportService = inject(ReportService);

  ngOnInit() {
    // ngOnInit is NOT an injection context, so pass destroyRef explicitly.
    this.reportService.data$
      .pipe(takeUntilDestroyed(this.destroyRef))
      .subscribe(data => this.render(data));
  }

  private render(data: unknown) { /* ... */ }
}
```

3.2 `DestroyRef.onDestroy` for non-RxJS cleanup

```
import { Component, DestroyRef, ElementRef, inject, OnInit } from '@angular/core';

@Component({
  selector: 'app-draggable-card',
  template: `<div #card class="card">Drag me</div>`,
})
export class DraggableCardComponent implements OnInit {
  private el = inject(ElementRef<HTMLElement>);
  private destroyRef = inject(DestroyRef);

  ngOnInit() {
```

```
    const node = this.el.nativeElement;
    const onPointerDown = (e: PointerEvent) => this.startDrag(e);

    node.addEventListener('pointerdown', onPointerDown);

    // Third-party widget with an explicit teardown API.
    const chart = new SomeChartLib(node.querySelector('.chart-target'));

    // Native timer.
    const pollId = setInterval(() => this.refreshLiveData(), 5000);

    // Register ALL cleanup in one place, tied to component destruction —
    // no ngOnDestroy method needed, and this works even in services/directives
    // that don't implement OnDestroy.
    this.destroyRef.onDestroy(() => {
      node.removeEventListener('pointerdown', onPointerDown);
      chart.destroy();
      clearInterval(pollId);
    });
  }

  private startDrag(_e: PointerEvent) { /* ... */ }
  private refreshLiveData() { /* ... */ }
}
```

3.3 Before/after leak fix

Before — leaking component:

```
@Component({ selector: 'app-notifications-badge', template: `{{ count }}` })
export class NotificationsBadgeComponent implements OnInit {
  count = 0;
  private sub?: Subscription;

  constructor(private notifications: NotificationsService) {} // root singleton

  ngOnInit() {
    // notifications.stream$ is built from a root-scoped Subject that lives
    // for the whole app session. This subscribe() callback closure captures
    // `this`, so as long as the Subject has this subscriber, the component
    // (and its entire view/DOM subtree) cannot be garbage collected —
    // even after the user navigates away and Angular destroys the view.
    this.sub = this.notifications.stream$.subscribe(n => (this.count = n));

    window.addEventListener('focus', this.refresh); // never removed
  }

  refresh = () => this.notifications.refresh();

  // No ngOnDestroy at all.
}
```

Symptom in production: navigating between the "inbox" route and other routes 50 times leaves 50 detached `NotificationsBadgeComponent` view trees in a heap snapshot, each still receiving live notification pushes and doing pointless change detection work.

After — fixed:

```
@Component({ selector: 'app-notifications-badge', template: `{{ count }}` })
export class NotificationsBadgeComponent implements OnInit {
  count = 0;
  private notifications = inject(NotificationsService);
  private destroyRef = inject(DestroyRef);

  ngOnInit() {
    this.notifications.stream$
      .pipe(takeUntilDestroyed(this.destroyRef))
      .subscribe(n => (this.count = n));

    const onFocus = () => this.notifications.refresh();
    window.addEventListener('focus', onFocus);
    this.destroyRef.onDestroy(() => window.removeEventListener('focus', onFocus));
  }
}
```

Result: on destroy, `takeUntilDestroyed` auto-unsubscribes (severing the strong reference from the singleton service's `Subject.observers` array back to the component), and the `focus` listener on `window` is removed, severing the other GC-root-to-component chain. The component becomes unreachable and is collected on the next GC cycle.

3.4 Heap-snapshot-guided debugging walkthrough (described via code + comments)

```
// Step 0: Reproduce.
// In the running app, open DevTools -> Memory tab -> select "Heap snapshot".
//
// Step 1: Baseline.
//   - Navigate to a neutral route (e.g., the app shell/home page).
//   - Click the trash-can icon ("Collect garbage") to force a GC pass.
//   - Take Snapshot #1 ("baseline").
//
// Step 2: Stress the suspect flow.
//   - Navigate to /inbox (mounts NotificationsBadgeComponent), then navigate
//     away to /home (destroys it). Repeat this cycle 10 times via the UI
//     (or drive it with Protractor/Playwright if you want exact repeatability).
//
// Step 3: Force GC + second snapshot.
//   - Click "Collect garbage" again.
//   - Take Snapshot #2 ("after 10 cycles").
//
// Step 4: Compare.
//   - Select Snapshot #2, switch the view dropdown from "Summary" to
//     "Comparison" against Snapshot #1.
```

```
//   - Sort by "# Delta" descending. Look for:
//       NotificationsBadgeComponent   Delta: +10   <-- exactly matches cycle count
//       (compiled template) / LView   Delta: +10 (or more, for child views)
//       Detached HTMLSpanElement      Delta: +10
//   - A delta that scales 1:1 with the number of navigation cycles is the
//     signature of a leak (a healthy component would show Delta: 0 or 1,
//     since Angular reuses/collects the rest).
//
// Step 5: Root-cause via Retainers.
//   - Click one of the leaked NotificationsBadgeComponent instances.
//   - Expand the "Retainers" panel at the bottom of the Summary/Comparison view.
//   - Walk the chain, e.g.:
//       NotificationsBadgeComponent
//         <- context (closure) in "subscribe" callback
//         <- Subscriber
//         <- observers[] (array)
//         <- Subject
//         <- NotificationsService (root singleton)
//         <- (root) ApplicationRef / Injector
//   - This chain confirms: the singleton service's Subject.observers array
//     is the retaining root. Fix: unsubscribe on destroy (see 3.3 "After").
//
// Step 6: Re-verify.
//   - Apply the fix, rebuild, repeat steps 1-4.
//   - Expected result: Delta for NotificationsBadgeComponent and its LView
//     should return to 0 (or 1, accounting for the currently-mounted instance).
```

4. Internal Working

4.1 Mark-and-sweep, concretely

V8 (and other JS engines) use a **mark-and-sweep** (generational, incremental/concurrent in practice) garbage collector, not reference counting:

1. **Roots**: the GC starts from a fixed set of roots — the global object (`window`), currently executing call stack frames/local variables, active closures, and pending timer/microtask/event-listener callback references held by the engine's internal task queues.
2. **Mark phase**: starting from each root, the collector traverses every reachable reference transitively (object → its properties → their properties → ...) and marks each visited object as "alive."
3. **Sweep phase**: any object in the heap that was **not** marked is considered garbage and its memory is reclaimed.
4. Because this is **reachability-based, not "does something logically still need it,"** an object with zero *conceptual* owners but one forgotten reference (e.g., left in an array, a closure, or an event-listener table) is still marked alive and will never be swept. This is precisely the mechanism behind every leak in section 2 — there is no bug in the GC; the leak is a bug in the program's reference graph.
5. V8 additionally distinguishes young generation (new/short-lived objects, collected frequently and cheaply via **Scavenge**) from old generation (long-lived objects promoted after surviving a few young-gen collections, collected less often via **Mark-Sweep-Compact**, which is more expensive and can cause visible pauses/jank — a reason unbounded leaks eventually manifest as stutter, not just rising memory).

4.2 Why subscriptions, closures, and detached DOM specifically block collection

- **RxJS subscription**: `Subject` / `Observable` internally stores each subscriber (an `Observer` / `Subscriber` wrapping your callback) in an internal `observers` array (or similar internal list depending on RxJS version). Your `.subscribe(cb)` callback is a closure that captured `this` (the component) in its lexical scope. As long as the array entry exists, the closure exists; as long as the closure exists, `this` is reachable; as long as `this` is reachable, the whole component instance (and everything it references — `@ViewChild`, injected services, and via the component's `LView`, its DOM nodes) is reachable. `unsubscribe()` removes the entry from that array, severing the chain.
- **Closures generally**: a JS closure retains a reference to its entire enclosing lexical scope (or the V8-optimized subset it actually uses), not just the specific variable you think you're using. A callback passed to `setInterval` / `addEventListener` /a third-party API that references `this.someMethod()` keeps the whole `this` alive for as long as the browser/engine holds that callback (i.e., for `setInterval`, forever until `clearInterval`; for a listener, forever until `removeEventListener`).
- **Detached DOM nodes**: Angular removing a node from the visible tree (structural directive toggling, `ComponentRef.destroy()`) only unlinks it from `document`'s DOM tree — it does not touch JS-side references. If any JS variable (a cached `ElementRef.nativeElement`, a jQuery selection stored on a service, a closure capturing the element for a resize/mutation observer) still points to that node, the node — and the DOM subtree/listeners attached to it — remains in the heap, invisible on screen but fully alive in memory. DevTools calls these out explicitly because a `Node` with no `document` ancestor but a nonzero retainer count is almost always unintentional.

4.3 How `DestroyRef` hooks into destruction

`DestroyRef` is an injectable handle scoped to the nearest enclosing "destroyable" context — a component, directive, embedded view, or the root environment injector. Internally:

- Every `LView` (Angular's internal per-view data structure) carries a list of destroy hooks (a `DestroyHookData` /callback array on the view's `TView` / `LView` cleanup slot).
- Calling `DestroyRef.onDestroy(callback)` pushes `callback` onto that view's cleanup list and returns an unregister function.
- When Angular actually destroys the view — via `ComponentRef.destroy()`, a structural directive removing an embedded view, router outlet deactivation, or parent view destruction cascading to children — the view-destroy code path (`destroyLView`) walks that cleanup list and invokes every registered callback synchronously, then proceeds to detach DOM nodes and clear internal references.
- `ngOnDestroy()` lifecycle methods are themselves implemented as one such registered destroy hook under the hood, meaning `DestroyRef.onDestroy` is a more general, composable version of the same mechanism — usable from services, directives, and functions called during construction (not just from a class that implements `OnDestroy`).
- `takeUntilDestroyed()` is implemented directly on top of `DestroyRef`: it calls `inject(DestroyRef)` (or uses the one passed explicitly), registers an `onDestroy` callback that calls `.unsubscribe()` /completes an internal `Subject` used to pipe through `takeUntil`, so the RxJS chain unsubscribes at exactly the same point `ngOnDestroy` would have fired.
- Because this is hook-list-based rather than polling/inheritance-based, it works uniformly across

components, directives, and even plain injectable services scoped to a component (`providers: [...]` on a component creates a child injector whose lifetime is tied to that component, and services there can inject their own local `DestroyRef`).

5. Edge Cases & Gotchas

- **Long-lived singleton services subscribing to short-lived component streams (the inverse leak).** Usually we worry about a component subscribing to a singleton's stream. The reverse is just as dangerous and easier to miss: a root service subscribes to an `@Output()` EventEmitter or an observable exposed by a component instance (`this.someComponent.valueChanges$.subscribe(...)` stored in the service). Because the service outlives the component, its subscription is the thing keeping the component alive — and since it's in a singleton, it silently accumulates one dead subscription per component instantiation for the life of the app. Fix: never let a singleton hold a subscription to a component-scoped stream without also unsubscribing when that specific component is destroyed (pass the component's own `DestroyRef` into the service call, or have the component push a teardown callback into the service).
- **Third-party libraries without a destroy method.** Not all libraries expose complete teardown. When a library only exposes partial cleanup (e.g., `.destroy()` removes its own DOM but leaves a global `ResizeObserver`/`window` listener registered internally), you must reach into whatever escape hatch is available (options to disable internal listeners, manually removing known event names, or — as a last resort — replacing the container element it was mounted into so old internal references become orphaned along with it). Always test with a heap snapshot after integrating a new third-party widget; don't assume `.destroy()` is complete just because it exists.
- **Angular's own `ChangeDetectorRef`/`ApplicationRef.tick()` retaining destroyed views.** If a destroyed component was manually attached via `ApplicationRef.attachView(viewRef)` (common in dynamic component creation, e.g., toast/modal services), forgetting to call `ApplicationRef.detachView(viewRef)` before `viewRef.destroy()` (or not destroying at all) means the view stays in `ApplicationRef`'s internal views list and keeps getting change-detected forever, alive.
- **Memory growth in SPAs that never fully reload.** Because there is no page reload between routes, every small leak compounds. A leak that "only" retains 50KB per navigation is invisible in a quick manual test but becomes a multi-hundred-MB problem after a day of a kiosk/dashboard app running unattended, or a power user who never closes the tab. This is why long-running Angular apps (dashboards, admin panels, kiosks) need explicit soak testing: script N thousand navigation cycles headlessly and assert heap size stabilizes rather than growing linearly.
- **Zone.js masking the real callback source.** Because Zone.js patches `setTimeout`/`addEventListener`/promises to trigger change detection, a leaked timer/listener still "works" (keeps firing, keeps triggering CD) even though its owning component is logically gone — there's no visible error, only silent CPU/memory waste, which is why these leaks are often caught only via profiling, never via functional bugs.
- **`takeUntilDestroyed()` called outside an injection context.** Calling it inside a `setTimeout` callback, a plain method not invoked during construction, or after the injection context has closed throws `NG0203` (or silently fails to resolve `DestroyRef`, depending on version) — the fix is always to either call it in the constructor/field initializer, or capture `DestroyRef` early (`private destroyRef = inject(DestroyRef)`) and pass it explicitly as shown in section 3.1.
- **Zoneless change detection does not remove the need for cleanup.** Removing Zone.js (signals-based/zoneless apps) means leaked timers/listeners no longer force spurious change detection cycles,

but the underlying memory leak (retained closures/DOM) is completely unaffected — zoneless apps leak exactly the same way; only one symptom (CPU churn) is masked.

6. Interview Questions & Answers

Q1: What is a memory leak in the context of a browser-based JS application, precisely?
A: It is not "the GC failing to run." It is a situation where an object that is logically no longer needed remains *reachable* from a GC root (global scope, active closures, timers, event listener tables) through some reference chain the developer didn't intend to keep alive. Since V8's mark-and-sweep collector only frees unreachable objects, any accidental live reference — however small — keeps the entire object graph behind it alive indefinitely.

Q2: Name the six most common Angular-specific memory leak sources.
A: (1) RxJS subscriptions to long-lived/singleton observables never unsubscribed; (2) native/Renderer2 DOM event listeners added but never removed; (3) detached DOM nodes still referenced by a closure or cached selector/service; (4) third-party library handles (charts, maps, editors) without a called `.destroy()`; (5) uncanceled `setInterval` / `setTimeout`; (6) singleton services holding arrays/maps/Subjects that reference destroyed component instances.

Q3: What's the difference between `takeUntil(this.destroy$)` and `takeUntilDestroyed()`?
Interviewer intent: Checking whether the candidate knows the modern API is a wrapper, not a different mechanism, and knows its constraints.
A: Functionally both unsubscribe an RxJS chain when the component is destroyed. `takeUntil` requires you to manually create a `Subject`, call `.next()` / `.complete()` in `ngOnDestroy`, and remember to add the operator to every stream — easy to forget on one pipe among several. `takeUntilDestroyed()` (Angular 16+, `@angular/core/rxjs-interop`) does the same thing internally but ties the teardown to the framework's own `DestroyRef` hook list, removing the boilerplate and the chance of forgetting `ngOnDestroy`. Constraint: it must be called within an injection context (constructor/field initializer) unless you explicitly pass a captured `DestroyRef` as an argument.

Q4: Why does the `async` pipe prevent leaks, and when should you still avoid relying on it?
A: The `async` pipe subscribes to the bound observable when its host view is created and calls `unsubscribe()` in its own `ngOnDestroy`, which fires automatically when the enclosing view is destroyed — so for purely template-display bindings it eliminates manual subscription management entirely. You should not rely on it when you need the emitted value in component *logic* (not just the template), when you need to combine/gate multiple streams before display, or when the subscription needs custom error handling/side effects beyond what template binding allows — in those cases use `takeUntilDestroyed()` / `DestroyRef` explicitly.

Q5: A singleton service exposes a `Subject`-backed stream. A component subscribes to it in `ngOnInit` and never unsubscribes. Walk through exactly why this leaks and what stays in memory.
Interviewer intent: Wants the retainer chain reasoning, not just "yes it leaks."
A: `Subject.subscribe(cb)` stores an internal `Subscriber` wrapping `cb` in the subject's `observers` array. `cb` is a closure created inside the component method, so it captures `this` (the component instance) in its lexical environment. The service is root-provided, so it — and its `Subject`, and the `observers` array, and every subscriber closure in it — lives for the entire application session. When the component's view is destroyed (route navigation away), Angular detaches its DOM and stops running its lifecycle, but the `Subject.observers` array still holds the subscriber closure, which still holds `this`. Therefore the component instance, its injected dependencies, its `@ViewChild` references, and (transitively, via the component's LView)

its detached DOM subtree all remain reachable from the root singleton and cannot be collected — repeated navigations create one such orphaned chain per visit, a linear leak.

Q6: How would you detect this specific kind of leak using Chrome DevTools, step by step?
A: Take a baseline heap snapshot after forcing GC on a neutral route. Perform the navigate-in/navigate-out cycle a fixed number of times (e.g., 10). Force GC again and take a second snapshot. Switch to the Comparison view between the two snapshots, sort by count delta, and look for the component's class name (and its `LView`/template class, and any "Detached" DOM element rows) showing a delta that matches the cycle count exactly — that 1:1 scaling is the leak signature. Click an instance, open the Retainers panel, and walk up the chain until you reach the actual root (in this case, the singleton service's `Subject.observers` array) to confirm and locate the fix.

Q7: What's the difference between `ngOnDestroy` and `DestroyRef.onDestroy`?
A: `ngOnDestroy` is a lifecycle interface method available only on classes recognized by Angular as components/directives/pipes/services with that hook, invoked once when Angular destroys that instance's view/injector. `DestroyRef.onDestroy(callback)` is the more general underlying primitive: an injectable handle that lets *any* code running in an injection context (including plain functions, other injectables, code inside a component's constructor) register an arbitrary cleanup callback against the nearest destroyable context, without needing to implement `OnDestroy` at all. `ngOnDestroy` itself is implemented as one particular registered destroy hook under the hood; `DestroyRef.onDestroy` lets you register as many as you like, from anywhere with access to the injector.

Q8: Why doesn't simply setting a component's fields to `null` in `ngOnDestroy` fix a leak?
A: Nulling a field on the component only removes the component's *own* outgoing reference to that object; it does nothing about the reference chain running the other direction — from some external root (a singleton service, a timer, `window`) into the component itself. If an external subscription/listener/timer still holds `this`, the whole component instance remains reachable regardless of what its internal fields point to; you have to break the *incoming* reference (unsubscribe, removeEventListener, clearInterval, remove from the service's registry), not just clear outgoing ones.

Q9: Explain mark-and-sweep and why it means "the GC can't be blamed" for a memory leak.
Interviewer intent: Tests real understanding of GC fundamentals versus cargo-culted "leak = bug" language.
A: Mark-and-sweep starts from a fixed set of roots (global object, call stack, active closures/timers/listener tables) and marks every object transitively reachable from them as alive; anything left unmarked after that traversal is swept (freed). The algorithm is deterministic and correct by definition — it will always collect everything that is truly unreachable, and never collect anything reachable. A "leak" is therefore never a GC malfunction; it's a program correctness issue where the developer created a reference chain from a long-lived root to an object they consider logically dead. Fixing a leak always means removing a reference from the graph (unsubscribing, deregistering a listener, clearing an array entry), never "forcing" the GC to try harder.

Q10: When would you reach for `WeakMap` or `WeakRef` in an Angular app, and what's the risk of misusing them?
A: `WeakMap` is appropriate when a service needs to associate metadata with component instances or DOM nodes (e.g., a cache keyed by `ElementRef.nativeElement`) without itself becoming a reason those objects stay alive — the map's keys don't count as strong references, so once all other references to a key are gone, that entry disappears too and its value becomes collectible. `WeakRef` is for holding an optionally-live reference you must explicitly `.deref()`-check, useful for building your own lifetime-aware caches or debugging tools. The risk: neither of these fixes an existing leak caused by a *strong* reference elsewhere (an active subscription, timer, or listener) — swapping a service's `Map` for a `WeakMap` while a subscription closure elsewhere still strongly references the component accomplishes nothing, since the component is still reachable via that other,

unrelated path. They're a memory *optimization* for cache-like structures, not a general leak-prevention mechanism, and `WeakRef` / `FinalizationRegistry` timing is non-deterministic, so neither should be used to trigger required cleanup logic.

Q11: A third-party charting library's `.destroy()` method removes its canvas but a global `window.resize` listener it registered internally keeps firing after the component is gone. How do you handle this?
A: First confirm via the library's docs/source whether `.destroy()` is documented as complete cleanup — many aren't. If it leaves internal listeners registered, options in order of preference: (1) check for a configuration option to disable the library's own auto-resize handling and instead drive resizing yourself via an Angular-managed `ResizeObserver` / `HostListener` that you *can* clean up; (2) if the library keeps a reference to the container element/canvas internally, replacing/removing that DOM node (rather than just calling `.destroy()`) can help orphan the internal closure along with it, though this isn't guaranteed if the library also holds a `window`-level reference independent of the DOM node; (3) as a last resort, monkey-patch or track the specific listener the library registers (if discoverable via source) and remove it manually in your own `DestroyRef.onDestroy` callback. Always verify with a heap snapshot that the fix actually breaks the retainer chain rather than assuming the library's public API is sufficient.

Q12: How would you soak-test an Angular SPA for memory leaks as part of CI, beyond manual DevTools inspection?
Interviewer intent: Wants to know if the candidate thinks about leak detection as a repeatable process, not just ad hoc debugging.
A: Drive the app with a headless browser (Playwright/Puppeteer) through a scripted loop of the suspect navigation/interaction cycle N times (e.g., 200 iterations of open-modal/close-modal or route A → route B). Periodically call the CDP `Performance` / `HeapProfiler` domain (`Page.getMetrics`, `HeapProfiler.takeHeapSnapshot`, or simpler: repeatedly sample `performance.memory.usedJSHeapSize` after forcing GC via `--js-flags=--expose-gc` and calling `global.gc()`, or via CDP `HeapProfiler.collectGarbage`) to record heap size at each checkpoint. Fit or simply eyeball the trend: a healthy app's heap size should plateau after an initial warm-up; a leak shows continued linear (or worse) growth across iterations. Fail the CI job if the growth-per-iteration exceeds a defined threshold. This catches regressions automatically instead of relying on someone remembering to open DevTools.

Q13: What happens if you register a `DestroyRef.onDestroy` callback from within a service injected in the root injector (not scoped to a component)?
A: Root-scoped services live for the entire application, and the root environment injector is generally only "destroyed" when the whole application is torn down (e.g., `ApplicationRef` destruction, or in tests via `TestBed`'s module teardown). So a root service's `DestroyRef` reflects the app's own lifetime, not any particular component's — the callback fires on full app shutdown, not on individual component navigations. This is a common mistake: injecting `DestroyRef` inside a singleton service and expecting it to fire per-component is wrong; you need the *component's* `DestroyRef` (passed into the service call, or the service must be provided at the component level via that component's `providers` array to get a component-scoped instance).

Q14: Detached DOM nodes show up in a heap snapshot with a nonzero "Distance" from GC roots but no parent in the visible document. What does that tell you, and what's your next debugging step?
A: It confirms the node has been structurally removed from the live document tree (so it's invisible/non-interactive on screen) but is still reachable from some JS reference — otherwise it would be swept and simply wouldn't appear in the snapshot at all. The next step is to open the Retainers panel for that node instance and walk the reference chain upward: look for whether it's held by a closure (an event handler or cached element

reference), an array/map in a service, or a third-party library's internal state. Once you identify the exact retaining object, trace it back to the code that created that reference and add the corresponding cleanup (removeEventListener, clearing the array entry, calling the library's destroy hook) at the appropriate destruction point (`ngOnDestroy` / `DestroyRef.onDestroy`).

7. Quick Revision Cheat Sheet

- **Leak = reachability, not GC failure.** Mark-and-sweep only frees objects unreachable from roots (window, stack, closures, timers, listener tables). Fixing a leak always means removing a reference, never "helping" the GC.
- **Six classic Angular leak sources:** unmanaged RxJS subscriptions to long-lived streams; native/Renderer2 listeners without removal; detached DOM retained by closures/cached selectors; third-party library handles without `.destroy()` calls; uncanceled `setInterval` / `setTimeout`; singleton services holding arrays/maps referencing destroyed components.
- **Prevention toolkit:** `async` pipe for template-only bindings (auto-unsubscribes with the view); `takeUntilDestroyed()` for programmatic subscriptions (must be in injection context, or pass `DestroyRef` explicitly); `DestroyRef.onDestroy()` for arbitrary non-RxJS cleanup (listeners, timers, third-party teardown); manual `unsubscribe()` in `ngOnDestroy` still valid for simple/legacy cases.
- `DestroyRef` **mechanism:** every `LView` carries a destroy-hook list; `onDestroy()` pushes a callback onto it; Angular invokes all hooks synchronously when the view is destroyed; `ngOnDestroy` and `takeUntilDestroyed` are both built on this same primitive.
- **Heap snapshot workflow:** baseline snapshot (force GC) → stress cycle N times → force GC → second snapshot → Comparison view sorted by delta → 1:1 delta-to-cycle-count is the leak signature → Retainers panel to find the exact root cause chain.
- `WeakMap` / `WeakRef` **:** use for component/DOM-node-keyed caches that shouldn't prevent collection; never a substitute for unsubscribing/removing listeners — a strong reference elsewhere still leaks regardless of what a `WeakMap` does. `FinalizationRegistry` timing is non-deterministic — diagnostic use only, never load-bearing cleanup.
- **Watch for the inverse leak:** singleton services subscribing to component-scoped streams/outputs leak just as badly as the reverse, and accumulate silently since the service never dies.
- **Zoneless doesn't fix leaks:** removing Zone.js only removes the CPU-churn symptom of a leaked timer/listener; the retained-memory problem is identical.
- **SPAs never reload:** small per-navigation leaks compound over long-running sessions (dashboards/kiosks) — soak-test with scripted navigation loops and heap-size sampling in CI, not just manual spot checks.

Created By - Durgesh Singh